Projet Graphes et Optimisation Combinatoire

Qubit allocation problem

Jérôme Rouzé

Faculté polytechnique de Mons
Université de Mons

UMONS POLYTECH MONS

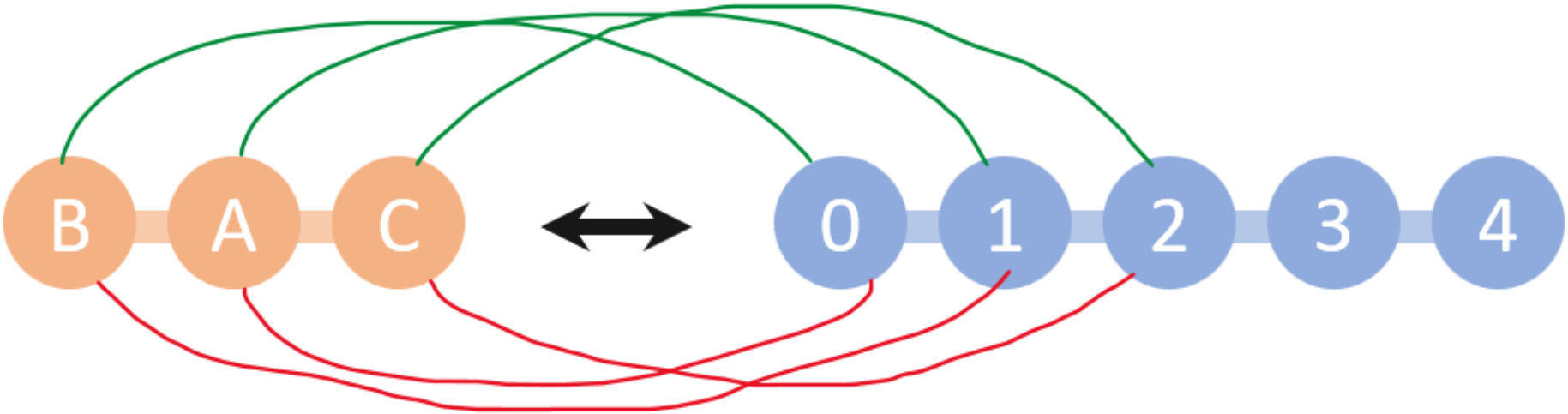
Présentation abstraite : problème de permutation partielle

Soit $n \leqslant m$ deux entiers naturels non nuls. On cherche le "meilleur" arrangement de n éléments parmi m.

- But du projet : Mettre un place un algorithme d'optimisation pour minimiser une fonction objectif "black box".
- La fonction objectif : Prend en entrée une liste **ordonnée** de n entiers deux à deux distincts dans $\{0, \ldots, m-1\}$ et renvoie un coût.
- Exemple : avec $m = 5$, $n = 3$, $[0, 3, 1]$ et $[1, 0, 3]$ sont deux solutions différentes admissibles. $[0, 3, 3]$, $[0, 3]$, $[0, 3, 1, 4]$ ne le sont pas.

Les instances du problème et la fonction objectif seront fournies !

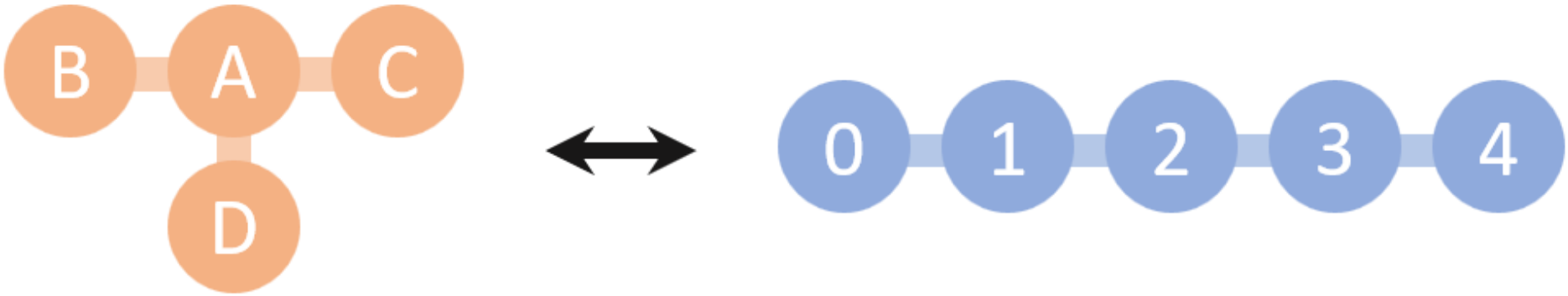
Application : Association de graphes



But : Associer les sommets du graphe orange à ceux du graphe bleu en préservant au mieux les arêtes oranges.

- Association $A \to 1; B \to 0; C \to 2$: aucun changement (association **parfaite**)
- Association $A \to 0; B \to 1; C \to 2$: plusieurs changements (association **imparfaite**)

Application : Association de graphes



But : Associer les sommets du graphe orange à ceux du graphe bleu en préservant au mieux les arêtes oranges.

- Pas d'association **parfaite**. Quelle est l'association qui minimise les changements (la moins imparfaite) ?

On attribue un coût à chaque changement et on cherche à minimiser le coût cumulé de chaque changement. Exemple d'application : le **Qubit Allocation Problem**.

Introduction : Qu'est-ce qu'un qubit ?

- La superposition

En informatique classique : un bit = 0 ou 1; deux bits : 00, 01, 10 ou 11.
En informatique quantique : un qubit = $\alpha|0\rangle + \beta|1\rangle$, où $\alpha, \beta \in \mathbb{C}$ tel que $|\alpha|^2 + |\beta|^2 = 1$
deux qubits = $\alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$

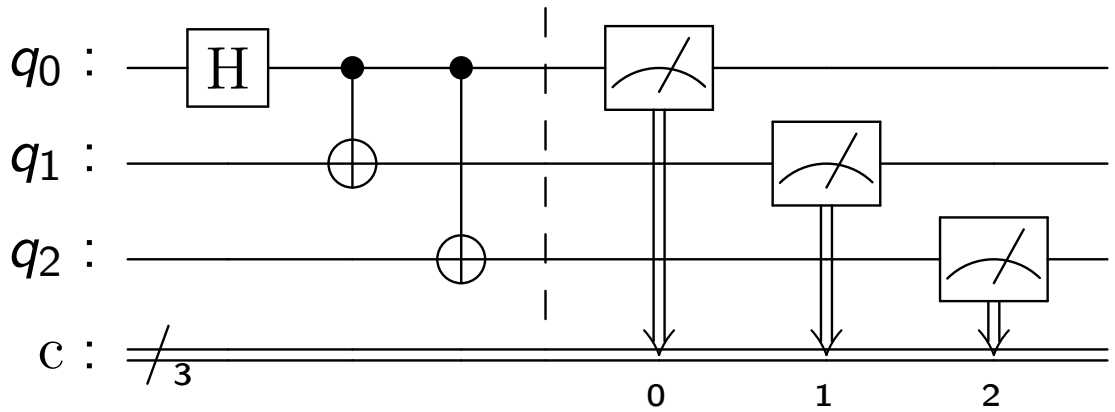
- L'intrication (entanglement)

Mathématiquement, c'est un état qui n'est pas le produit tensoriel de deux états. En pratique, c'est un état d'un système où plusieurs particules quantiques sont dans des états liés.

Exemple : Etat de Bell : $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$

Modèle à porte (gates-based model)

Dans ce modèle, un programme (ou circuit) quantique est une suite de portes quantiques ($\approx$ portes logiques). Un exemple de circuit :

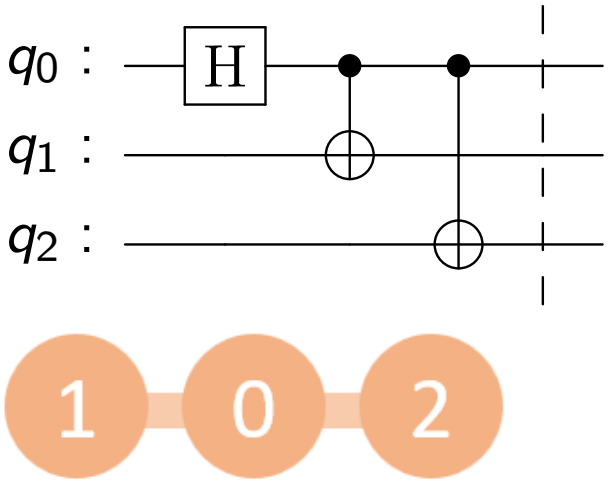


Porte H = porte de Hadamard, crée la superposition $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$

Porte CNOT (Controlled NOT), applique NOT à la cible si le contrôle est en $|1\rangle$ et ne fait rien sinon. Crée de l'intrication.

Lien avec l'association de graphes

Connectivité qubits virtuels



$\xrightarrow{?}$

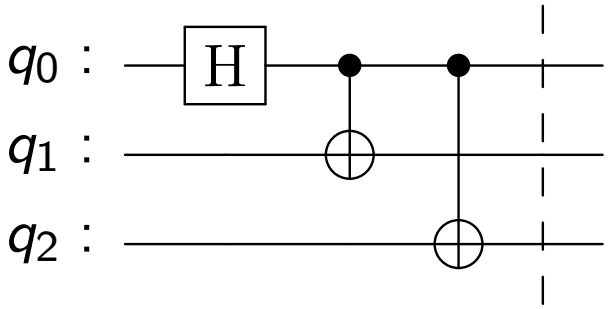
Connectivité qubits physiques



D'un circuit quantique, on tire un graphe orange à associer au graphe bleu, qui représente les connexions entre les qubits physiques d'un ordinateur quantique. **Le coût des changements correspondent aux portes quantiques à ajouter au circuit pour l'exécuter malgré une association imparfaite.** On parle de "routing" du circuit.

Un exemple pour comprendre

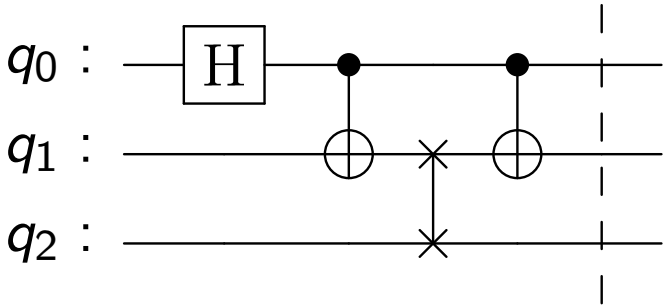
Le circuit original



Connectivité qubits physiques



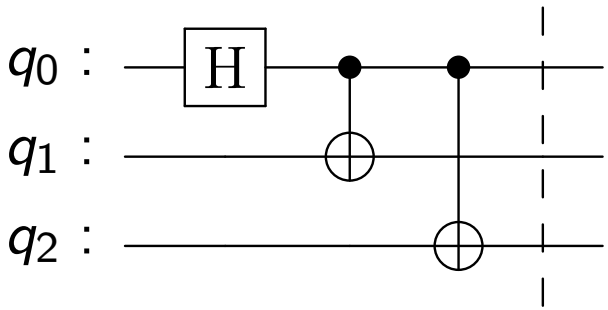
Association [0,1,2] **imparfaite**



Longueur = 6 (Longueur(swap)=3)

Un exemple pour comprendre

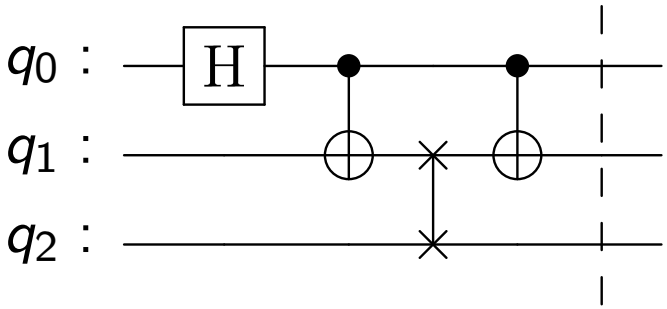
Le circuit original



Connectivité qubits physiques

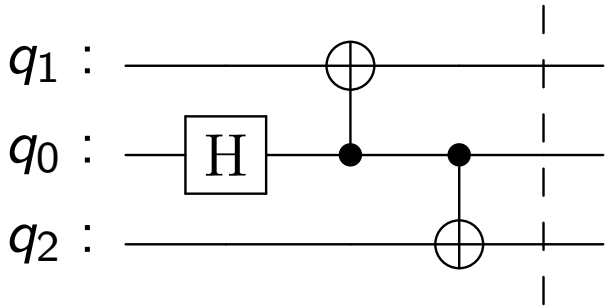


Association [0,1,2] **imparfaite**



Longueur = 6 (Longueur(swap)=3)

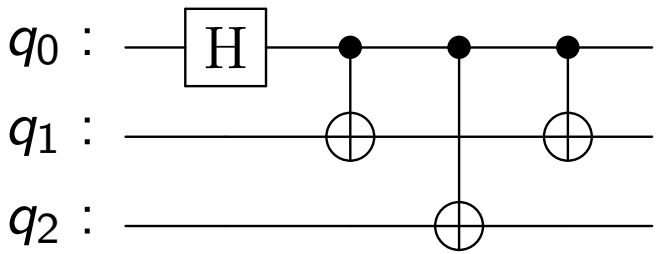
Association [1,0,2] **parfaite**



Longueur = 3, solution optimale

Différence avec l'association de graphes

Prenons le circuit :





Ici, on a encore la même connectivité qu'avant, mais la connection 0–1 est double par rapport à la connection 0–2. Pour autant le problème n'est pas l'association d'un graphe pondéré. En effet, les deux associations 0–1 ne sont pas nécessaire **en même temps** dans le circuit. Il est donc possible de casser la seconde connection 0–1 pour la reconstruire ensuite.
"Association de graphes" et "Qubit Allocation Problem" sont donc similaires mais différents.

Retour au projet

Sur Moodle se trouve les codes StudentCode.py et source.py. Ce dernier fournie la fonction objectif et quelques fonctions utilitaires pour charger les instances du problème. **source.py doit ne pas être modifié.**

- La fonction objectif calcule les changements nécessaires et renvoie le coût. Elle prend en entrée **une liste ordonnée de n entier parmi** $\{0, \ldots, m-1\}$, n et m dépendant de l'instance (voir slide suivante).
- Le choix de l'instance se fait en donnant son numéro à la fonction `instance_selector`.
- Ne rien changer d'autres !
- Pensez à installer la librarie Qiskit à l'aide la commande `pip install qiskit == 1.2.4`.
  La version 1.2.4 est importante, ne changer pas de version !

Utilisation de StudentCode.py

```
16  ##-------------------------------------------------------
17  ##     Pour choisir une instance:
18  ##     Modifier instance_num ET RIEN D'AUTRE
19  ##     La fonction instance_selector revoie n et m
20  ##     ainsi que "backend" et "circuit" qui sont
21  ##     utiliser dans la fonction objectif (voir fitness)
22  ##-------------------------------------------------------
23  instance_num=1     #### Entre 1 et 9 inclue
24
25  backend,circuit,n,m = instance_selector(instance_num)
26
27  ##-------------------------------------------------------
28  ##     A vous de jouer !
29  ##-------------------------------------------------------
30
31  ###### Votre code ici
```

En modifiant la variable instance_num, on change d'instance. Il ne faut rien modifier d'autres au dessus de l'espace "A vous de jouer !". Les variables n et m qui dépendent de l'instance sont chargées au choix de l'instance. Pas besoin de les entrer à la main !.

Utilisation de StudentCode.py – Rendu

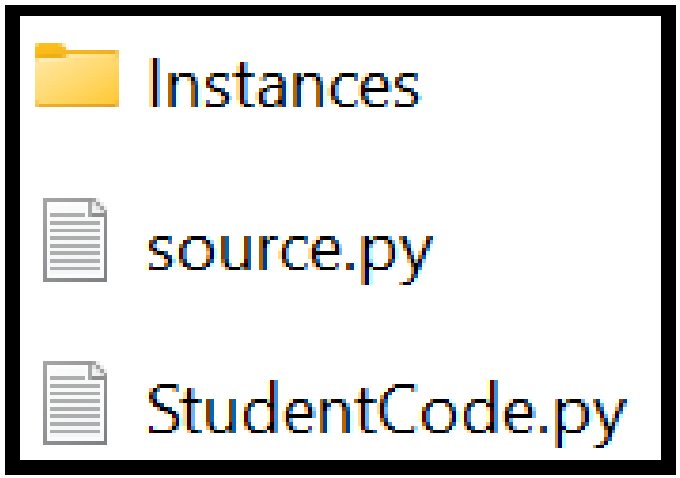
Ce qu'on attend de vous pour vérifier vos solutions du challenge:

- A la fin de l'execution de votre métaheuristique : écrire l'association puis son coût dans un fichier nommé [GROUPE]_Challenge_[instance_number].txt.

Exemple : Le groupe 18 trouve que la solution [1, 0, 2] a pour coût 53 pour l'instance Challenge 0. Ils rendent alors un fichier Groupe18_Challenge_0.txt contenant exactement : 1 0 2 53

Les instances

- Les instances se trouve dans le dossier "Instances" sur Moodle, à placer dans le même répertoire que StudentCode.py et source.py.
- StudentCode.py contient un test pour être sûr tout est correctement installé.
- Exemple d'espace de travail:



Les instances – Meilleures solutions connues

Instance	n	m	Coût de la meilleure solution connue
1	12	16	50
2	16	16	59*
3	10	16	108
4	19	20	35
5	12	20	323
6	14	20	25
7	20	27	22*
8	20	27	17
9	25	27	51

* Solution exacte